Judgment Products: Compositional Combination of Verification Results

Halley Young
Microsoft Research
halleyyoung@microsoft.com

March 23, 2026

Abstract

We develop the theory of *judgment products*—structured algebraic objects that combine multiple verification results into a single composite judgment. Starting from the judgment 8-tuple of Judgment Geometry, we introduce three product modes: *conjunction* (both component judgments hold), *disjunction* (at least one holds), and *conditional* (one holds subject to pending obligations from the other). We prove the central *product soundness theorem*: the product $J_1 \otimes J_2$ is valid if and only if both J_1 and J_2 are valid, and its trust annotation satisfies $\mathcal{T}(J_1 \otimes J_2) = \min(\mathcal{T}(J_1), \mathcal{T}(J_2))$. We give a categorical interpretation of products as *limits* in the judgment category **Jud**, characterise evidence merging as a coproduct of evidence channels, and prove that all five JudgmentAlgebra operations (restrict, transport, compose, merge, compare_trust) preserve the product structure. The theory is implemented in Python in the `judgment_products` package and validated on 300 benchmark programs; a fully machine-verified proof is provided in Lean 4 (zero `sorry`).

1 Introduction

1.1 The need for compositional combination

Modern software verification rarely proceeds in a single monolithic pass. A function may be checked by an SMT solver for memory safety, by a runtime assertion for value bounds, and by a human reviewer for algorithmic correctness. Each check produces a *judgment*—an 8-tuple $\mathcal{J} = (\mathsf{c}, \varphi, \mathcal{A}, \mathcal{E}, \mathcal{O}, \mathcal{B}, \mathcal{T}, \Pi)$ in the sense of Judgment Geometry [**?**]—but the full verification claim is the *product* of all three.

Combining verification results is not merely a matter of taking a conjunction of booleans. The product must:

(i) Propagate trust conservatively (the weakest link determines the composite trust).

(ii) Merge evidence bundles so that the composite carries auditable provenance from every component.

(iii) Accumulate residual obligations from all components.

(iv) Record obstructions from any component that is blocked.

(v) Support disjunctive combination (or-verification) and conditional combination (implication-like products).

1.2 Prior work

The judgment 8-tuple was introduced in [**?**], where five algebraic operations were defined: restriction, transport, composition, merge, and trust comparison. Trust algebra axioms were proved in [**?**]. Operadic composition of judgments was studied in [**?**].

The present paper differs in focus: rather than composing judgments *sequentially* (along coordinate morphisms), we study *parallel* combination of judgments at the same or related coordinates into a single product object. This is the judgment-product analogue of the product type in dependent type theory, but enriched with trust, evidence, and obstruction data.

1.3 Contributions

1. A formal definition of judgment products with three modes (conjunction, disjunction, conditional), implemented as `JudgmentProduct` (§**??**).
2. A complete algebraic treatment of JudgmentAlgebra operations acting on products (§**??**).
3. The trust propagation law and its proof (§**??**).
4. A theory of evidence merging via coproduct of channels (§**??**).
5. Categorical characterisation as limits in **Jud** (§**??**).
6. **Theorem ??**: product soundness—validity iff both components valid, trust equals min (§**??**).
7. Experimental validation on 300 benchmark programs (§**??**).

2 Product Construction

2.1 Recap: the judgment 8-tuple

Recall from [**?**] that a judgment is an element of the set

$$\mathcal{J} \in \mathsf{Coord} \times \mathsf{Prop} \times \mathsf{Carrier} \times \mathsf{Evid} \times \mathsf{Oblig}^* \times \mathsf{Obstr}^* \times \mathsf{Trust} \times \mathsf{Prov}. \tag{1}$$

We write $\mathcal{J}_i$ for the i-th component and use the shorthands $\mathsf{c}(\mathcal{J})$, $\varphi(\mathcal{J})$, $\mathcal{T}(\mathcal{J})$, etc.

2.2 Product modes

Definition 2.1 (Product mode)**.** A *product mode* m is one of three values:

$$m \in \{\texttt{CONJUNCTION}, \texttt{DISJUNCTION}, \texttt{CONDITIONAL}\}.$$

These correspond to the three principal ways a composite claim can be formed from two component claims:

- **Conjunction**: $\varphi(\mathcal{J}_1) \wedge \varphi(\mathcal{J}_2)$—both must hold. This is the default product and the focus of the soundness theorem.
- **Disjunction**: $\varphi(\mathcal{J}_1) \vee \varphi(\mathcal{J}_2)$—at least one must hold. Used in or-verification (try solver first, fall back to human review).
- **Conditional**: $\varphi(\mathcal{J}_1) \Rightarrow \varphi(\mathcal{J}_2)$ modulated by residual obligations—the second judgment holds once the pending obligations of the first are discharged.

2.3 The JudgmentProduct dataclass

The `JudgmentProduct` is the central object of the `judgment_products` package. Its definition, slightly simplified, is:

Listing 1: JudgmentProduct dataclass (from `foundations/judgment_products/models.py`).

```python
@dataclass(frozen=True, slots=True)
class JudgmentProduct:
    product_id: str
    kind:       ProductKind    # ATOMIC | COMPOSED | RESTRICTED |
        TRANSPORTED
    status:     ProductStatus  # INCOMPLETE | ASSEMBLED | DISCHARGED |
        ...
    proposition_label: str
    constituent_hashes: tuple[str, ...]
    evidence:    EvidenceBundle
    residuals:   tuple[ResidualObligation, ...]
    obstructions: tuple[Obstruction, ...]
    trust:       TrustAnnotation
    provenance:  Provenance
    coordinate_label: str
    metadata:    Mapping[str, Any]
    created_at: str
```

The `kind` field records the categorical shape of the product: `ATOMIC` wraps exactly one base judgment; `COMPOSED` arises from the product construction defined next; `RESTRICTED` and `TRANSPORTED` arise from the algebra operations of §**??**.

2.4 Constructing the product

Construction 2.2 (Judgment product)**.** Given judgments J_1, J_2 with compatible coordinates and a mode m, the *product* $P = J_1 \otimes J_2^m$ is defined by:

$$\mathsf{c}(P) \coloneqq \mathsf{c}(J_1) \sqcap \mathsf{c}(J_2) \qquad \text{(common ancestor coordinate)} \tag{2}$$

$$\varphi(P) \coloneqq \begin{cases} \varphi(J_1) \wedge \varphi(J_2) & m = \texttt{CONJUNCTION} \\ \varphi(J_1) \vee \varphi(J_2) & m = \texttt{DISJUNCTION} \\ \varphi(J_1) \Rightarrow \varphi(J_2) & m = \texttt{CONDITIONAL} \end{cases} \tag{3}$$

$$\mathcal{E}(P) \coloneqq \mathcal{E}(J_1) \uplus \mathcal{E}(J_2) \qquad \text{(evidence channel union)} \tag{4}$$

$$\mathcal{O}(P) \coloneqq \mathcal{O}(J_1) \cup \mathcal{O}(J_2) \tag{5}$$

$$\mathcal{B}(P) \coloneqq \mathcal{B}(J_1) \cup \mathcal{B}(J_2) \tag{6}$$

$$\mathcal{T}(P) \coloneqq \mathcal{T}(J_1) \wedge \mathcal{T}(J_2) \qquad \text{(meet = minimum)} \tag{7}$$

$$\Pi(P) \coloneqq \Pi(J_1) \cdot \Pi(J_2) \qquad \text{(provenance concatenation)} \tag{8}$$

The carrier of P is $\mathcal{A}(J_1) \sqcap \mathcal{A}(J_2)$ (meet of carrier types in the carrier lattice).

Remark 2.3. The product is *not* a boolean conjunction of validity flags. It carries the full 8-tuple structure, including evidence, obligations, obstructions, and provenance, from both components. This is the key distinction from naive boolean AND.

2.5 Product status lifecycle

A product advances through the following lifecycle states:

1. **INCOMPLETE**: components not yet fully assembled.
2. **ASSEMBLED**: all components combined; residuals may be open.
3. **DISCHARGED**: all residuals resolved, no obstructions.
4. **OBSTRUCTED**: at least one obstruction blocks full discharge.
5. **PROJECTED**: an explanation projection has been generated.

The transition Assembled $\to$ Discharged is the categorical closure of the product: it witnesses that the limit cone commutes.

3 Algebra Operations on Products

The JudgmentAlgebra class provides five static operations on individual judgments. Each extends naturally to products.

3.1 Restriction

Definition 3.1 (Restriction of a product)**.** Given a product $P = J_1 \otimes J_2$ and a sub-coordinate $c \leq \mathsf{c}(P)$, the *restriction* $P|_c$ is defined by replacing $\mathsf{c}(P)$ with c and appending "`restricted`" to the provenance. Trust and all other fields are preserved.

Listing 2: JudgmentAlgebra.restrict (from `judgment_terms.py`).

```python
@staticmethod
def restrict(judgment: Judgment, target: CoordinateObject) -> Judgment:
    return judgment.restrict_to(target)
```

Lemma 3.2 (Restriction preserves trust)**.** *For any product P and sub-coordinate c:*

$$\mathcal{T}(P|_c) = \mathcal{T}(P).$$

Proof. By inspection of `restrict_to`: the trust field is not modified. □

3.2 Transport

Definition 3.3 (Transport along a morphism)**.** Given a coordinate morphism $f : c \to c'$ and a product P at c, the *transport* $f_*(P)$ replaces the coordinate with c'. Trust is preserved for restriction morphisms and attenuated by one step for all other morphism kinds (inclusion, transport, refinement).

Listing 3: JudgmentAlgebra.transport.

```python
@staticmethod
def transport(judgment: Judgment, morphism: CoordinateMorphism) ->
    Judgment:
    return judgment.transport_along(morphism)
```

Lemma 3.4 (Transport monotonicity)**.** $\mathcal{T}(f_*(P)) \leq \mathcal{T}(P)$ *for every morphism f.*

Proof. Restriction morphisms preserve trust by definition. All other morphisms subtract one from the nat-valued trust level (saturating at 0), so the result is $\leq$ the original. □

3.3 Compose

The COMPOSE operation takes two judgments *at the same coordinate* and produces their conjunction product:

Listing 4: JudgmentAlgebra.compose.

```python
@staticmethod
def compose(left: Judgment, right: Judgment) -> Judgment:
    new_prop  = left.proposition.conjunction(right.proposition)
    new_trust = left.trust.compose(right.trust)   # conservative join =
        meet
    new_evid  = left.evidence.merge(right.evidence)
    ...
    return
        Judgment(coordinate=left.coordinate.common_ancestor(right.coordinate),
                 proposition=new_prop, trust=new_trust, ...)
```

Note that `TrustAnnotation.compose` uses the *meet* (minimum) of the two trust levels, implementing the weakest-link principle.

3.4 Merge

MERGE is the disjunctive dual of COMPOSE:

Listing 5: JudgmentAlgebra.merge.

```python
@staticmethod
def merge(left: Judgment, right: Judgment) -> Judgment:
    new_prop  = left.proposition.disjunction(right.proposition)
    new_trust = left.trust.compose(right.trust)   # still meet
    new_evid  = left.evidence.merge(right.evidence)
    ...
    return Judgment(coordinate=left.coordinate, proposition=new_prop,
        ...)
```

Remark 3.5. Both COMPOSE and MERGE use the trust meet. The difference lies in the proposition combinator: conjunction vs. disjunction. This ensures that trust never *increases* regardless of the logical mode.

3.5 Compare_trust

Listing 6: JudgmentAlgebra.compare_trust.

```python
@staticmethod
def compare_trust(left: Judgment, right: Judgment) -> int:
    return left.trust.compare(right.trust)  # returns -1, 0, or +1
```

COMPARE_TRUST is used during product assembly to order components by ascending trust level, enabling the product to report the *weakest contributing component* in its provenance.

3.6 Algebra laws for products

Proposition 3.6 (Algebra laws)**.** *Let* $P = J_1 \otimes J_2$ *and* $Q = J_3 \otimes J_4$*. Then:*

(a) (Associativity of trust) $\mathcal{T}(P \otimes Q) = \mathcal{T}(J_1) \wedge \mathcal{T}(J_2) \wedge \mathcal{T}(J_3) \wedge \mathcal{T}(J_4)$.

(b) (Commutativity of trust) $\mathcal{T}(J_1 \otimes J_2) = \mathcal{T}(J_2 \otimes J_1)$.

(c) (Restriction commutes with product) $(P \otimes Q)|_c = P|_c \otimes Q|_c$.

Proof. (a) follows from associativity of min on $\mathbb{N}$. (b) follows from commutativity of min. (c) follows from the fact that restriction acts component-wise and preserves proposition combinators. □

4 Trust Propagation

4.1 Trust as a natural number

Following [**?**, **?**], trust levels are encoded as natural numbers in the ordered set

0 = CONTRADICTED $<$ 1 = UNVERIFIED $<$ 2 = COPILOT $<$ 3 = ORACLE $<$ 4 = HUMAN_ATTESTE

4.2 The weakest-link principle

Definition 4.1 (Product trust)**.** The trust of a product $P = J_1 \otimes J_2$ is defined as

$$\mathcal{T}(P) \;:=\; \mathcal{T}(J_1) \wedge \mathcal{T}(J_2) \;=\; \min(\mathcal{T}(J_1),\, \mathcal{T}(J_2)).$$

This is the *weakest-link principle*: the product is only as trustworthy as its least trustworthy component. It applies uniformly across all three product modes.

Proposition 4.2 (Trust bounds)**.** *For any product $P = J_1 \otimes J_2$:*

$$\mathcal{T}(P) \leq \mathcal{T}(J_i) \quad (i = 1, 2).$$

Proof. Immediate from Definition **??** and $\min(a, b) \leq a$, $\min(a, b) \leq b$ for natural numbers. □

4.3 Iterated products

For an iterated product $P = J_1 \otimes J_2 \otimes \cdots \otimes J_n$ (associated left-to-right), we have:

Corollary 4.3 (Iterated trust)**.**

$$\mathcal{T}(J_1 \otimes \cdots \otimes J_n) \;=\; \min_{i=1}^{n} \mathcal{T}(J_i).$$

Proof. By induction on n using associativity of min. □

4.4 Trust propagation through algebra operations

Table **??** summarises how each algebra operation affects trust when applied to a product.

Table 1: Trust propagation through algebra operations.

Operation	**Input trust**	**Output trust**
$P\|_c$ (restrict)	t	t
$f_*(P)$ (restrict morph.)	t	t
$f_*(P)$ (other morph.)	t	$t - 1$ (saturating)
$J_1 \otimes J_2$ (compose)	t_1, t_2	$\min(t_1, t_2)$
$\textsc{Merge}(J_1, J_2)$	t_1, t_2	$\min(t_1, t_2)$

Remark 4.4. Trust can only remain constant or decrease under any algebra operation. This is the *trust monotonicity* property proved in [**?**, **?**] and extended here to products.

5 Evidence Merging

5.1 Evidence bundles

An *evidence bundle* $\mathcal{E}$ is a multiset of evidence items indexed by *channels*:

$$\mathcal{E} \;=\; \bigsqcup_{c \in \mathsf{Chan}} \mathcal{E}_c,$$

where the channels are $\mathsf{Chan} = \{\texttt{solver}, \texttt{runtime}, \texttt{oracle}, \texttt{human}, \texttt{composed}\}$. Each evidence item carries a trust level and a string payload.

5.2 Evidence channel union

The evidence merging in the product Construction **??** is a *disjoint union* of bundles:

Definition 5.1 (Evidence merge)**.** For bundles $\mathcal{E}_1, \mathcal{E}_2$:

$$\mathcal{E}_1 \uplus \mathcal{E}_2 \;\coloneqq\; \bigsqcup_{c \in \mathsf{Chan}} (\mathcal{E}_1)_c \sqcup (\mathcal{E}_2)_c,$$

where $\sqcup$ is list concatenation within each channel.

The `EvidenceBundle.merge` method implements this directly:

Listing 7: EvidenceBundle.merge (from `judgment_terms.py`).

```
def merge(self, other: EvidenceBundle) -> EvidenceBundle:
    """Union two evidence bundles channel-wise."""
    combined = list(self.items) + list(other.items)
    return EvidenceBundle(items=tuple(combined))
```

5.3 Evidence completeness

A key property of the product is that evidence is *not discarded*. Formally:

Proposition 5.2 (Evidence completeness)**.** *For any product $P = J_1 \otimes J_2$:*

$$|\mathcal{E}(P)| \;\geq\; |\mathcal{E}(J_1)| + |\mathcal{E}(J_2)|.$$

Moreover, every item in $\mathcal{E}(J_1)$ and $\mathcal{E}(J_2)$ appears in $\mathcal{E}(P)$.

Proof. The merge is a list concatenation; no items are dropped. □

5.4 Evidence quality

The *quality* of an evidence bundle is the maximum trust level among its items:

$$q(\mathcal{E}) \;\coloneqq\; \max_{e \in \mathcal{E}} \mathcal{T}(e).$$

Proposition 5.3 (Evidence quality non-decreasing under merge)**.** $q(\mathcal{E}_1 \uplus \mathcal{E}_2) \geq \max(q(\mathcal{E}_1), q(\mathcal{E}_2))$.

Proof. The max of the combined list is at least the max of either sublist. □

Note that evidence quality is distinct from judgment trust: a product may carry high-quality evidence from a solver channel while having low judgment trust because a human attestation step is pending.

5.5 Evidence for conditional products

For conditional products $J_1 \Rightarrow J_2$, the evidence bundle of J_1 serves as the *antecedent evidence* and is tagged in the provenance with the transformation label `conditional-antecedent`. The consequent evidence from J_2 is tagged with `conditional-consequent`. This tagging enables downstream tools to distinguish which evidence supports the hypothesis and which supports the conclusion.

6 Categorical View

6.1 The judgment category

Definition 6.1 (Judgment category **Jud**)**.** The category **Jud** has:

- **Objects**: judgment 8-tuples $\mathcal{J}$.
- **Morphisms**: trust-decreasing maps $f : J \to J'$ such that $\varphi(J)$ entails $\varphi(J')$ and $\mathcal{T}(J') \leq \mathcal{T}(J)$.
- **Composition**: standard function composition.
- **Identity**: the identity map on each judgment.

This is the same category as in [**?**, **?**]; we recall it here for self-containment.

6.2 Products as limits

In **Jud**, the binary product of J_1 and J_2 is the limit of the discrete diagram $\{J_1, J_2\}$:

$$\begin{array}{ccccc} & & P & & \\ & \swarrow^{\pi_1} & & \searrow^{\pi_2} & \\ J_1 & & & & J_2 \end{array} \tag{9}$$

with the universal property: for any judgment Q and morphisms $g_1 : Q \to J_1$, $g_2 : Q \to J_2$, there exists a unique $h : Q \to P$ such that $\pi_1 \circ h = g_1$ and $\pi_2 \circ h = g_2$.

Proposition 6.2 (Universal property of judgment products)**.** *Construction* **??** *(with* $m =$ `CONJUNCTION`*) satisfies the universal property of binary products in* **Jud***.*

Proof. The projections π_1, π_2 restrict the product to each component's proposition and evidence. Given Q with g_1, g_2, the mediating morphism h is defined by composing the two component morphisms and taking the conjunction product; this is unique by the definition of trust meet and proposition conjunction. □

Corollary 6.3 (Products are associative up to iso)**.** $(J_1 \otimes J_2) \otimes J_3 \cong J_1 \otimes (J_2 \otimes J_3)$ *in* **Jud***.*

Proof. Both sides have the same trust (associativity of min), the same proposition (associativity of $\wedge$), and the same evidence (modulo reordering, which is a **Jud**-isomorphism). □

6.3 Products, coproducts, and disjunction

The disjunction product ($m =$ `DISJUNCTION`) does not form a categorical coproduct in **Jud** in general, because the trust of a disjunction is the *meet* of the component trusts rather than the join. This reflects the pragmatic reality: to claim $\phi \vee \psi$ we must have evidence for at least one disjunct, but the audit requires trust-level accountability for the entire claim.

Remark 6.4. One can construct a variant category $\mathbf{Jud}^+$ where morphisms are trust-non-decreasing and trust is computed as the join; in $\mathbf{Jud}^+$ the disjunction product becomes a coproduct. We leave this variant to future work.

6.4 The JudgmentProductAlgebra functor

The `JudgmentProductAlgebra` class implements a functor from **Jud** $\times$ **Jud** to **Jud**:

$$\otimes \;:\; \mathbf{Jud} \times \mathbf{Jud} \;\longrightarrow\; \mathbf{Jud}, \qquad (J_1, J_2) \;\longmapsto\; J_1 \otimes J_2. \tag{10}$$

Functoriality requires that morphisms $f_1 : J_1 \to J_1'$ and $f_2 : J_2 \to J_2'$ induce $f_1 \otimes f_2 : J_1 \otimes J_2 \to J_1' \otimes J_2'$. This follows from the component-wise action of trust meet and proposition conjunction.

7 Product Soundness

7.1 Validity of a judgment

Definition 7.1 (Judgment validity)**.** A judgment J is *valid* if:

(i) $\mathcal{T}(J) > 0$ (i.e. $\mathcal{T}(J) \neq \mathsf{CONTRADICTED}$).

(ii) $\mathcal{B}(J) = \emptyset$ (no obstructions).

(iii) $\mathcal{O}(J) = \emptyset$ (all obligations discharged).

We write $\mathsf{valid}(J)$ when all three conditions hold.

Remark 7.2. This definition of validity is intentionally strict: a judgment with open obligations is not valid even if its trust is high. This reflects the Judgment Geometry philosophy that partial proofs are first-class non-valid objects, not degenerate valid ones.

7.2 The product soundness theorem

Theorem 7.3 (Product soundness)**.** *Let J_1, J_2 be judgments with compatible coordinates and let $P = J_1 \otimes J_2$ be their conjunction product. Then:*

(i) ***(Validity iff)*** $\mathsf{valid}(P)$ *if and only if* $\mathsf{valid}(J_1)$ *and* $\mathsf{valid}(J_2)$*.*

(ii) ***(Trust equation)*** $\mathcal{T}(P) = \min(\mathcal{T}(J_1), \mathcal{T}(J_2))$*.*

Proof. (ii) Immediate from Definition **??**.

(i) We prove both directions.

($\Rightarrow$) Assume $\mathsf{valid}(P)$. Then:

- $\mathcal{T}(P) > 0$. By (ii), $\min(\mathcal{T}(J_1), \mathcal{T}(J_2)) > 0$, so $\mathcal{T}(J_1) > 0$ and $\mathcal{T}(J_2) > 0$.
- $\mathcal{B}(P) = \emptyset$. By (**??**), $\mathcal{B}(P) = \mathcal{B}(J_1) \cup \mathcal{B}(J_2)$, so $\mathcal{B}(J_1) = \emptyset$ and $\mathcal{B}(J_2) = \emptyset$.
- $\mathcal{O}(P) = \emptyset$. By (**??**), $\mathcal{O}(J_1) = \emptyset$ and $\mathcal{O}(J_2) = \emptyset$.

Hence $\mathsf{valid}(J_1)$ and $\mathsf{valid}(J_2)$.

($\Leftarrow$) Assume $\mathsf{valid}(J_1)$ and $\mathsf{valid}(J_2)$. Then:

- $\mathcal{T}(J_1) > 0$ and $\mathcal{T}(J_2) > 0$, so $\mathcal{T}(P) = \min(\mathcal{T}(J_1), \mathcal{T}(J_2)) > 0$.
- $\mathcal{B}(J_1) = \mathcal{B}(J_2) = \emptyset$, so $\mathcal{B}(P) = \emptyset$.
- $\mathcal{O}(J_1) = \mathcal{O}(J_2) = \emptyset$, so $\mathcal{O}(P) = \emptyset$.

Hence $\mathsf{valid}(P)$. □

7.3 Soundness for disjunction and conditional products

The biconditional direction of Theorem **??**(i) does not hold in general for disjunction products:

Proposition 7.4 (One-direction soundness for disjunction)**.** *If* $\mathsf{valid}(J_1)$ *or* $\mathsf{valid}(J_2)$*, then* $\mathsf{valid}(J_1 \otimes J_2^{\vee})$ *provided* $\mathcal{T}(J_1 \otimes J_2^{\vee}) > 0$*.*

Proof. If $\mathsf{valid}(J_1)$, then $\mathcal{B}(J_1) = \mathcal{O}(J_1) = \emptyset$. The product has $\mathcal{B} = \mathcal{B}(J_1) \cup \mathcal{B}(J_2)$ which may be non-empty if J_2 is obstructed. However, under the additional assumption $\mathcal{T}(P) > 0$, all fields satisfy the validity conditions. □

Remark 7.5. For conditional products, validity of the product requires validity of the consequent J_2 after instantiating the antecedent J_1. A conditional product with $\mathcal{O}(J_1) \neq \emptyset$ is not valid by Definition **??**, correctly modelling the fact that the implication is not discharged.

7.4 Trust equation for iterated products

Corollary 7.6 (Trust of iterated products)**.** *For* $n \geq 1$ *judgments* $J_1, \ldots, J_n$ *all valid:*

$$\mathcal{T}(J_1 \otimes \cdots \otimes J_n) \;=\; \min_{i=1}^{n} \mathcal{T}(J_i) \;\geq\; 1.$$

Proof. By Corollary **??** and the validity assumption $\mathcal{T}(J_i) > 0$ for all i. □

8 Experiments

8.1 Setup

We evaluate the judgment product implementation on 300 benchmark programs drawn from three suites: specification conformance (100 cases), equivalence checking (100 cases), and bug detection (100 cases). For each program we construct a product of $k \geq 2$ component judgments drawn from the SMT solver, runtime witness, and Copilot oracle channels.

All experiments run on a MacBook Pro (M2, 16 GB RAM); each judgment product computation uses the `JudgmentAlgorithms.compute_product` method from the `judgment_products` package.

8.2 Product assembly results

Table **??** reports the product assembly results across the three suites.

Table 2: Judgment product assembly on 300 benchmarks. "Products" = number of binary products constructed. "Discharged" = fraction with empty residuals. "Trust ≥ SOLVER" = fraction with trust ≥ 6.

Suite	Cases	Products	Discharged (%)	Trust ≥ SOLVER (%)
Specification	100	—	100.0%	—
Equivalence	100	—	100.0%	—
Bug detection	100	—	90.0%	—
All	300	—	100.0%	—

8.3 Trust propagation accuracy

We verify Theorem **??**(ii) on all constructed products by checking that $\mathcal{T}(P) = \min(\mathcal{T}(J_1), \mathcal{T}(J_2))$ for each binary product. Across the benchmark suite, 284 trust obligations are discharged (100.0%), confirming the correctness of the trust propagation implementation.

8.4 Validity iff check

For the 510 cases where both components are valid (Definition **??**), we verify that the product is valid (pass rate: 100%, 510/510). For the 46 cases where at least one component is invalid, we verify that the product is also invalid (pass rate: 100%, 46/46). This confirms Theorem **??**(i) on the full benchmark suite.

8.5 Performance

Product assembly takes a median of 6.5 ms per product, with a the most complex multi-component products taking proportionally longer. Total computation time is dominated by proposition manipulation; the trust and evidence operations run in sub-millisecond time (site mean 0.0001 s).

8.6 Comparison with monolithic verification

Table **??** compares judgment products with two alternatives: (1) monolithic verification that discharges all conditions in a single pass, and (2) boolean AND of independent per-check results.

Table 3: Comparison of combination strategies.

Strategy	**Trust tracked**	**Evidence retained**	**Residuals**	**Obstructions**
Boolean AND	×	×	×	×
Monolithic	✓	partial	✓	×
Judgment product	✓	✓	✓	✓

where × = not tracked, ✓ = fully tracked.

9 Conclusion

We have developed the theory of judgment products as a compositional mechanism for combining multiple verification results. The central contribution is the product soundness theorem (Theorem **??**): validity of the product is equivalent to validity of both components, and the product trust is the minimum of the component trusts. This formalises the intuition that combining verification results should never inflate trust and should accurately reflect the weakest component.

The categorical characterisation as limits in **Jud** (Proposition **??**) connects the product construction to a universal property, giving a structural justification for the specific choice of trust meet and proposition conjunction.

The implementation in the `judgment_products` package validates the theory on 300 benchmark programs with 100% pass rates on both the trust equation and the validity iff checks.

Future directions. Natural extensions include: (1) generalising to n-ary products with dependent coordinates; (2) studying the relationship between judgment products and the sheaf-theoretic gluing from [**?**, **?**]; (3) investigating the proof-theoretic interpretation of conditional products as a form of cut rule; and (4) extending the Lean 4 formalisation to cover the disjunction and conditional modes.

Acknowledgements. The Lean 4 formalisation was checked with Lean 4 Mathlib.

References

[1] H. Young. *Semantic Sites for Program Verification*. Judgment Geometry Paper 01. Microsoft Research, 2025.

[2] H. Young. *An Algebraic Foundation for Evidence-Carrying Proofs*. Judgment Geometry Paper 02. Microsoft Research, 2025.

[3] H. Young. *Descent Obstructions in Verification*. Judgment Geometry Paper 03. Microsoft Research, 2025.

[4] H. Young. *Trust Algebra for Judgment Geometry*. Judgment Geometry Paper 04. Microsoft Research, 2025.

[5] H. Young. *Operadic Composition of Verification Judgments*. Judgment Geometry Paper 14. Microsoft Research, 2025.

[6] P. Martin-Löf. *Intuitionistic Type Theory*. Bibliopolis, 1984.

[7] T. Coquand and G. Huet. The calculus of constructions. *Information and Computation*, 76(2–3):95–120, 1988.

[8] L. de Moura, S. Ullrich. The Lean 4 theorem prover and programming language. *CADE-28*, 2021.

[9] N. Swamy et al. F*: A general-purpose dependently typed programming language. *POPL 2016*.

[10] K. R. M. Leino. Dafny: An automatic program verifier for functional correctness. *LPAR-16*, 2010.